

ChurnGuard: A Machine Learning-Powered Credit Card Customer Retention System

This presentation details the development and deployment of a machine learning application designed to predict credit card customer churn within the banking industry. Our goal is to enable proactive intervention strategies to retain valuable customers, thereby enhancing business sustainability.

JT by Joshua Zachary Tan

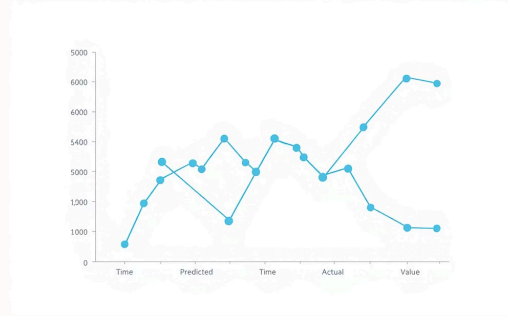


Introduction & Project Overview



Addressing Credit Card Churn

Customer churn in credit card services results in significant revenue loss, increased acquisition costs, and potential brand damage. Retaining existing customers is critical for profitability and business sustainability in the competitive banking sector.



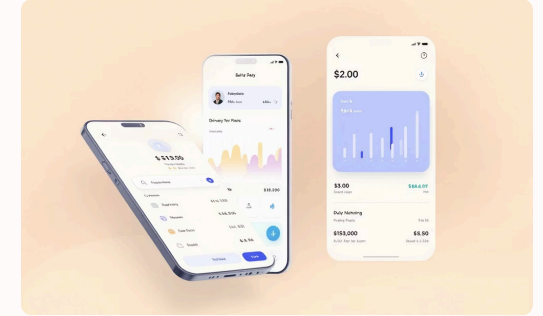
Developing Accurate Predictive Models

Our project focuses on creating highly accurate machine learning models to predict credit card customer churn. This enables proactive identification of at-risk customers before they depart.



Generating Actionable Insights

The system generates clear, actionable insights, empowering banks to design and implement targeted retention strategies that are both timely and effective, ensuring valuable customers are retained.



Seamless Business Integration

ChurnGuard is designed as a user-friendly application for seamless integration into existing banking operations. Its intuitive interface ensures that insights are easily accessible and actionable by business users.

Why Focus on Credit Card Churn?



Mitigate Revenue Loss

By identifying the probability of churn of potential churners, banks can implement targeted campaigns to prevent their departure, thereby safeguarding revenue streams.



Optimize Resource Allocation

The system outputs a churn probability score for each customer. Retention efforts can be prioritized towards customers based on risk level, ensuring efficient use of resources.



Enhance Customer Lifetime Value

Proactive retention strategies lead to longer customer relationships, increasing the overall lifetime value each customer brings to the institution.



Improve Customer Satisfaction

By addressing potential pain points before churn occurs, banks can improve customer satisfaction and loyalty.



```

graph TD
    Frontend[Frontend] --> API1[API]
    API1 --> BackendAPI[API]
    BackendAPI --> Database1[(Database)]
    Database1 --> Fromductor[Fromductor]
    Fromductor --> UPFJ[UPFJ Profodction]
    UPFJ --> Packer[Packer]
    Packer --> Database2[(Database by DAPS)]
    Database2 --> Dataend[Dataend]
  
```


- Interactive, multi-page web application
- Dashboard for churn trends and statistics
- Prediction input, result and LLM generation
- Historical churn predictions
- Risk and LLM Prompt Configuration interfaces

- RESTful API for all backend logic
- Churn prediction endpoints
- Prediction history management
- ML model integration

- Relational database (SQLite/PostgreSQL)
- Stores prediction history
- Maintains user configurations
- Ensures data persistence

Data Preprocessing Pipeline

Our robust data pipeline encompassed cleaning, numerical scaling, categorical encoding, and addressing class imbalance via SMOTE.

Initial Data Cleaning

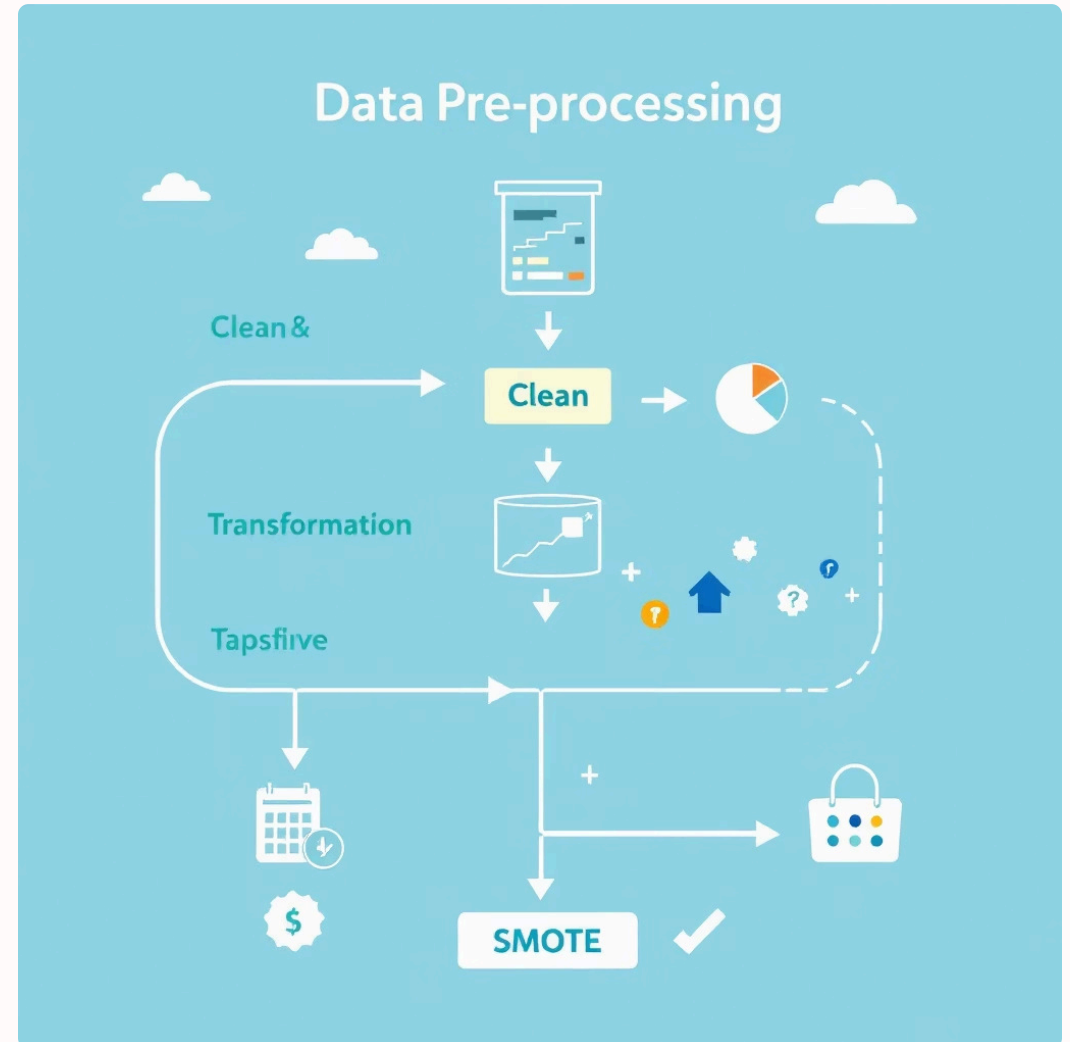
Dropped irrelevant columns ('RowNumber', 'CustomerId', 'Surname') and handled missing values with appropriate imputation strategies.

Feature Transformation

Applied StandardScaler to numerical features and OneHotEncoder to categorical features using scikit-learn's ColumnTransformer.

Class Imbalance Handling

Integrated SMOTE directly into the preprocessing pipeline using imblearn.pipeline to prevent data leakage.



Hyperparameter Tuning with GridSearchCV



Pipeline Integration

ImbPipeline combines SMOTE with models to prevent data leakage during cross-validation, maintaining preprocessing integrity across all evaluation folds.



Exhaustive Search

GridSearchCV systematically explores all possible hyperparameter combinations to identify optimal configuration for each model architecture.



Performance Evaluation

ROC AUC scoring metric ensures robust evaluation despite class imbalance, focusing on model's ability to distinguish potential churners.

Model-Specific Configurations

Model	Cross Validation	Parallel Jobs
Logistic Regression	5	n_jobs=-1 (all CPUs)
Random Forest	5	n_jobs=-1 (all CPUs)
Neural Network	3	n_jobs=1 (GPU optimized)

GridSearchCV eliminates manual trial-and-error by automating the hyperparameter optimization process. This systematic approach ensures we find the most effective model configuration for identifying potential credit card churners.

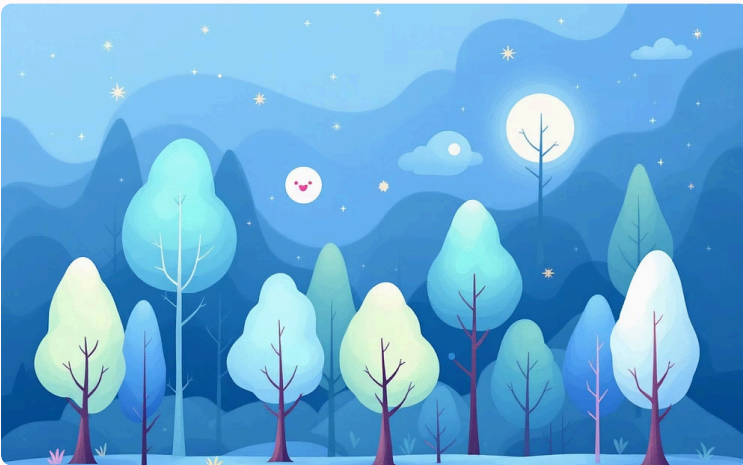
The neural network uses fewer cross-validation folds (3 vs 5) due to GPU compute constraints, balancing thoroughness with computational efficiency.

Machine Learning Models



Logistic Regression

Implemented as a baseline model due to its interpretability and efficiency. Provided foundational understanding of feature importance.



Random Forest Classifier

Chosen for its ensemble learning capabilities, robustness to overfitting, and strong performance on tabular data.



Custom PyTorch Neural Network

Developed to explore deep learning potential. Multi-layered perceptron with ReLU activations and sigmoid output for binary classification.

Logistic Regression Testing and Validation Results



Performance Metrics

Achieved 0.7115 ROC-AUC and 0.7059 Recall-score on test data, establishing a solid baseline while maintaining model interpretability.



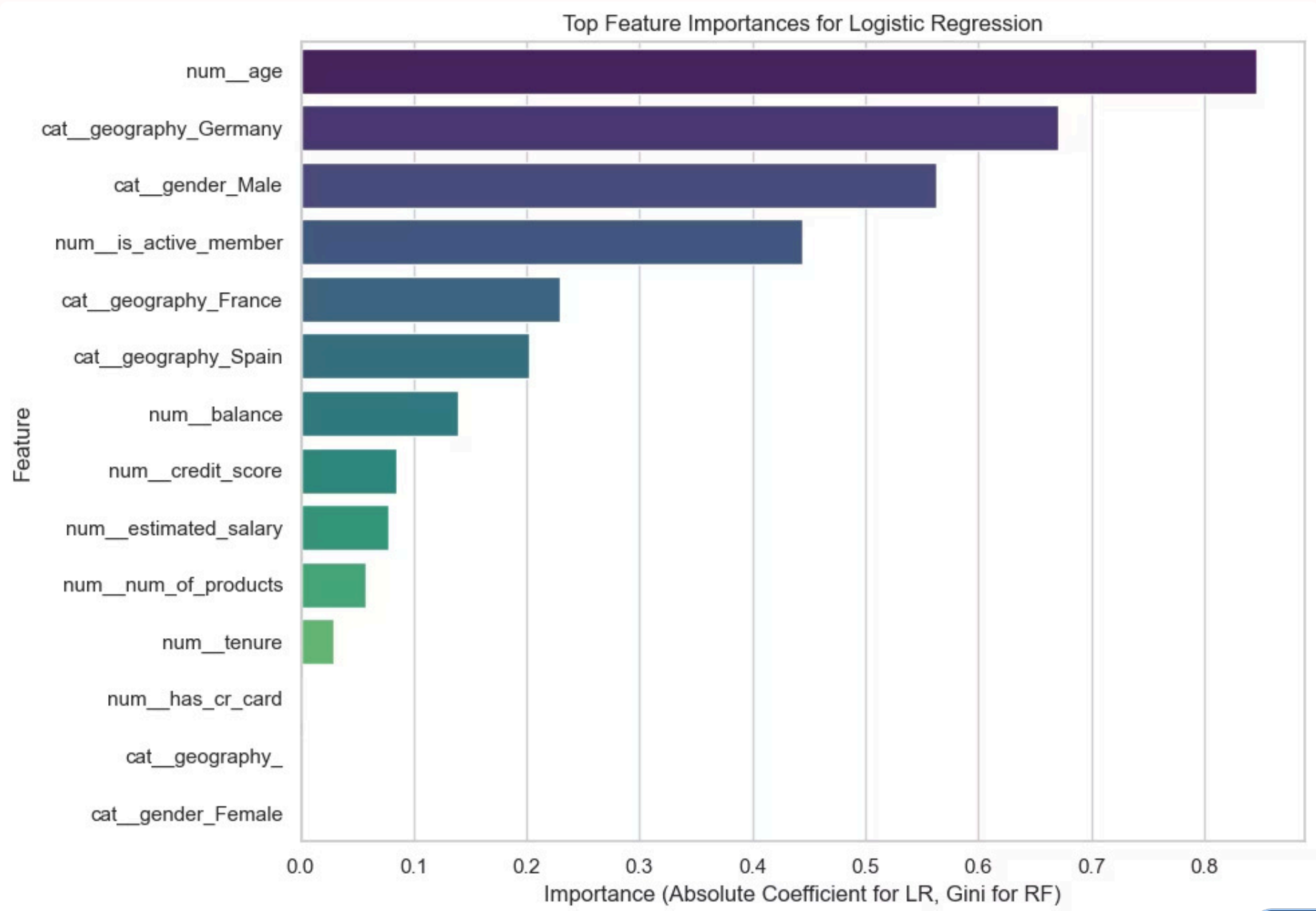
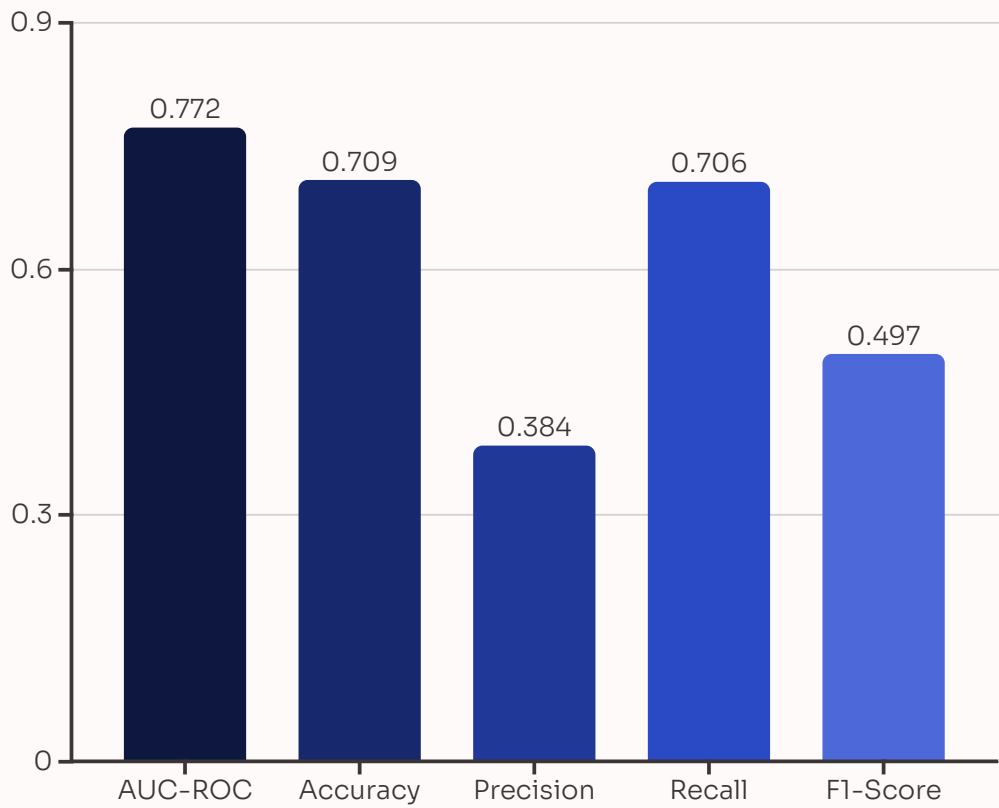
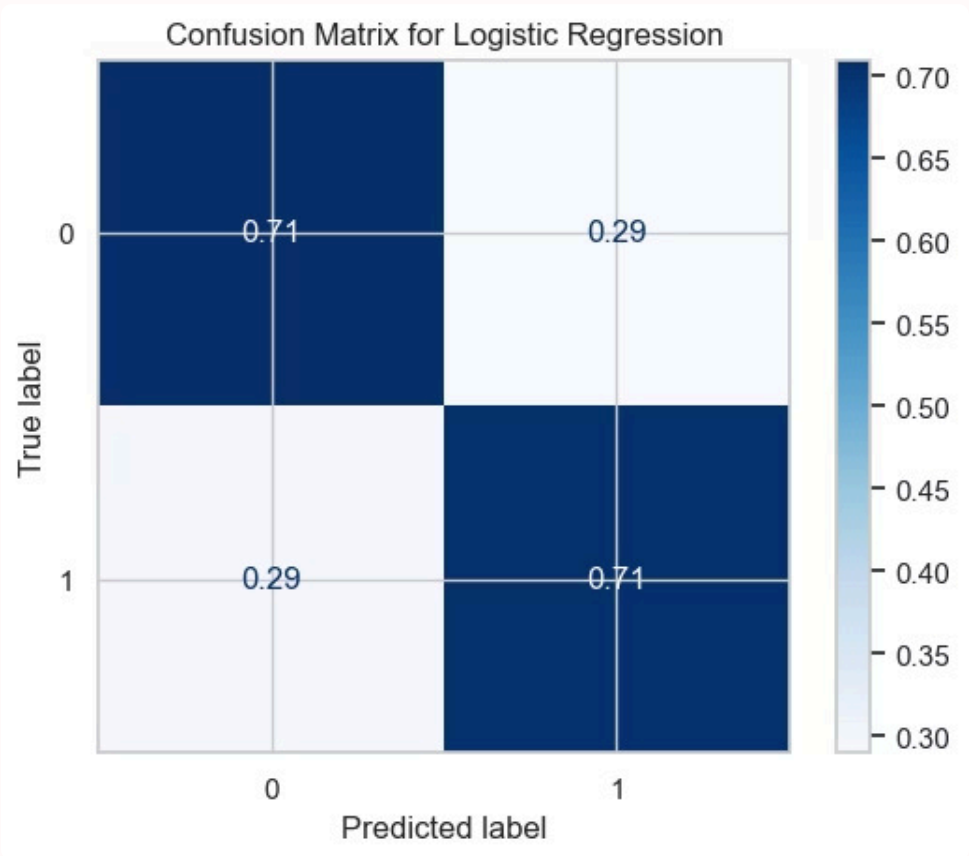
Feature Importance

Identified key churn indicators: Age, Geography (Germany) and Gender (Male) show strong correlation with churn likelihood



Conclusion

Logistic Regression performs well on **recall**, which is valuable for catching churners early, but **low precision** means a high number of false alarms.



Random Forest Testing and Validation Results



Performance Metrics

Achieved 0.8585 ROC-AUC and 0.6618 Recall-score on test data. 0.6143 on the F1-Score that shows a better trade-off between preicion and recall



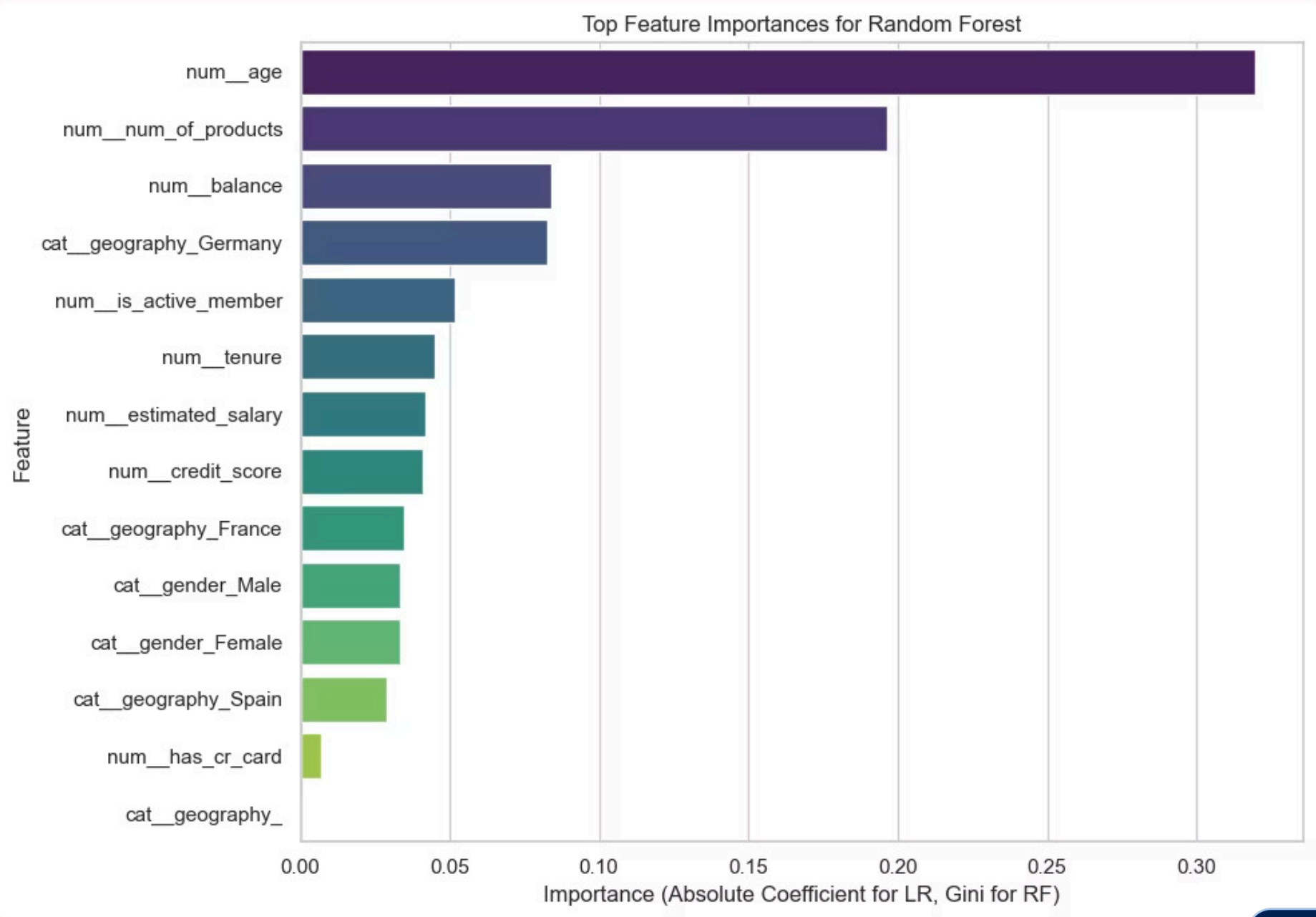
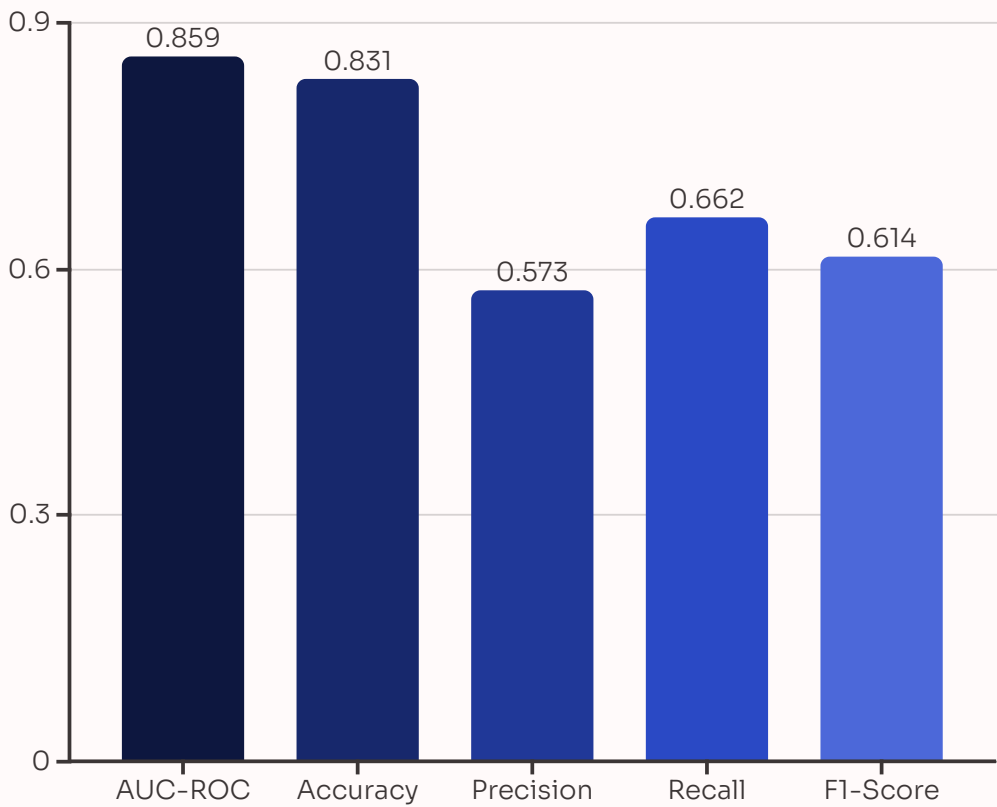
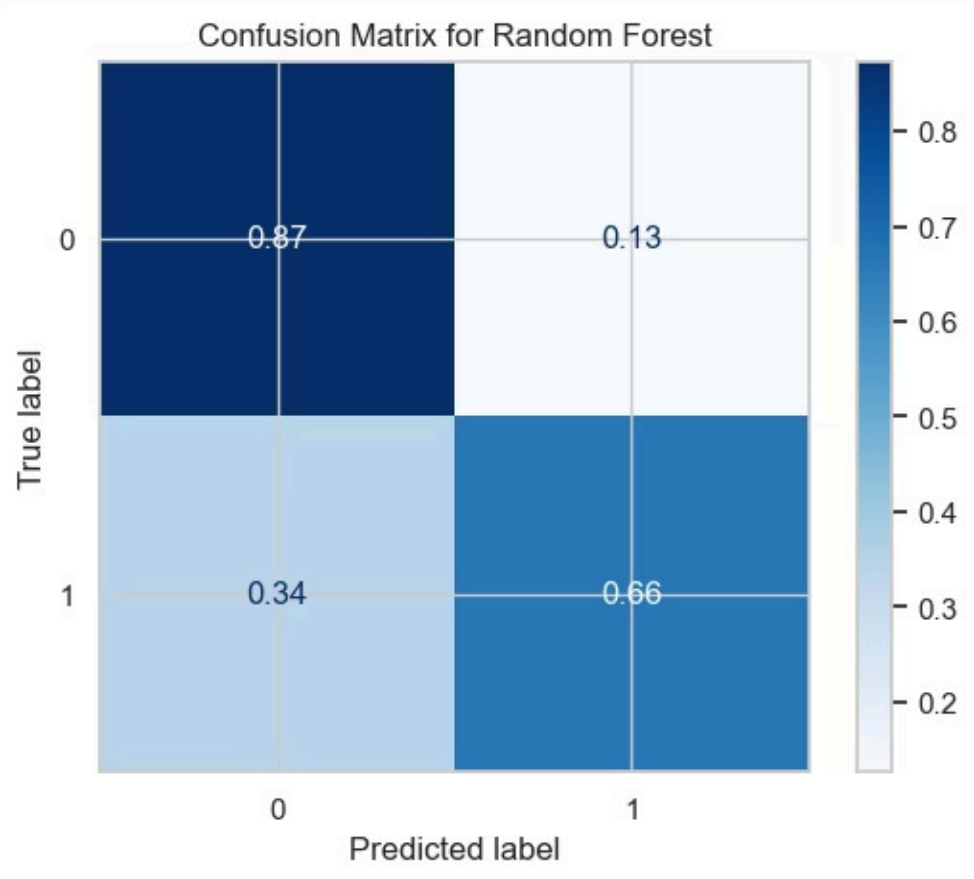
Feature Importance

Identified key churn indicators: Age and Number of Products



Conclusion

Random Forest provides the best overall performance, especially in **accuracy, precision,** and **F1-score**, with a solid **ROC AUC**. It balances false positives and false negatives better than the other models, making it ideal for production.



Neural Network Hyperparameter Tuning



The Neural Network underwent an exceptionally extensive hyperparameter tuning process using GridSearchCV with a carefully defined search space:

Hidden layer sizes: [(128, 64), (256, 128, 64)]

Multi-layer architectures allow the model to **learn complex, non-linear patterns** in customer behavior. **(128, 64)** is a compact network ideal for faster training with reduced overfitting risk. **(256, 128, 64)** offers greater learning capacity, better suited for capturing subtle churn signals, at the cost of longer training and potential overfitting.

Learning rates: [0.001, 0.0001]

0.001 is a standard starting point in many neural networks—fast convergence. **0.0001** offers finer updates for **stability**, especially when deeper networks or noisy gradients are involved.

Batch sizes: [32, 64]

Batch size of 32 is commonly used for its ability to strike a balance between speed and stability. **Batch size of 64** offers **faster training** with more stable gradients, useful on modern GPUs like the RTX 4060.

Dropout rates: [0.2, 0.5]

Dropout is a regularization technique that randomly "drops" neurons during training to prevent overfitting. **0.2** is conservative, retaining more information in each iteration. **0.5** is aggressive, pushing the network to generalize more.

Early stopping patience: [5, 10]

Prevents overtraining by monitoring validation loss and stopping when no significant improvement is seen. **Patience of 5** is useful for quicker termination when validation stagnates early. **Patience of 10** gives the model more time to escape local minima.

Min delta: [0.001]

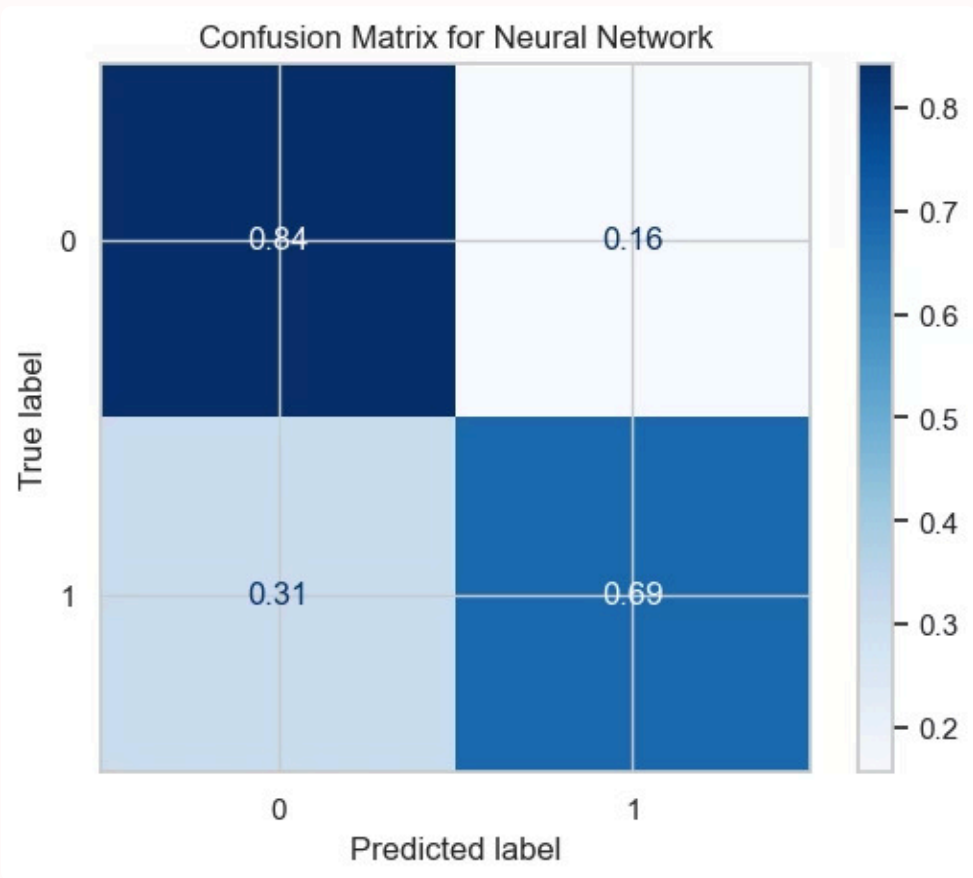
Ensures that only meaningful improvements in validation loss reset the patience counter. **0.001** is a sensible threshold that avoids stopping on noise while still capturing stagnation.

This rigorous search was calibrated for approximately 24 hours of training time on an NVIDIA RTX 4060 laptop, demonstrating significant commitment to finding the optimal configuration.

Neural Network Testing and Validation Results

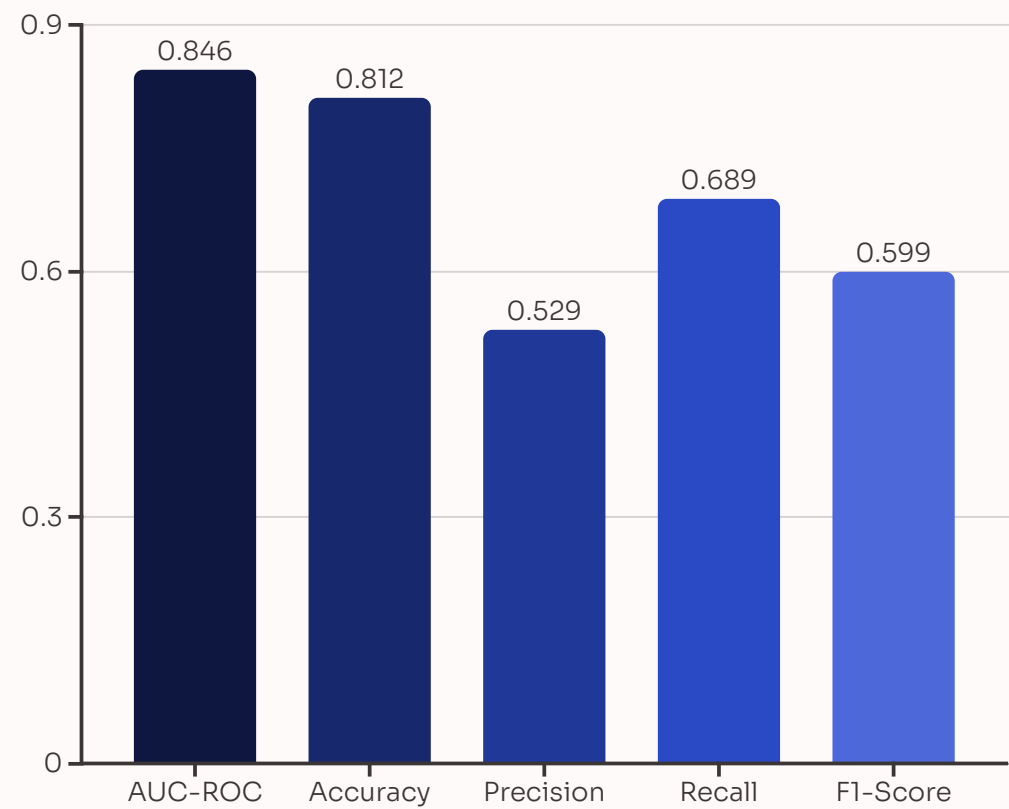
Performance Metrics

Achieved 0.8457ROC-AUC and 0.6618 Recall-score on test data. 0.6143 on the F1-Score that shows a better trade-off between preicion and recall

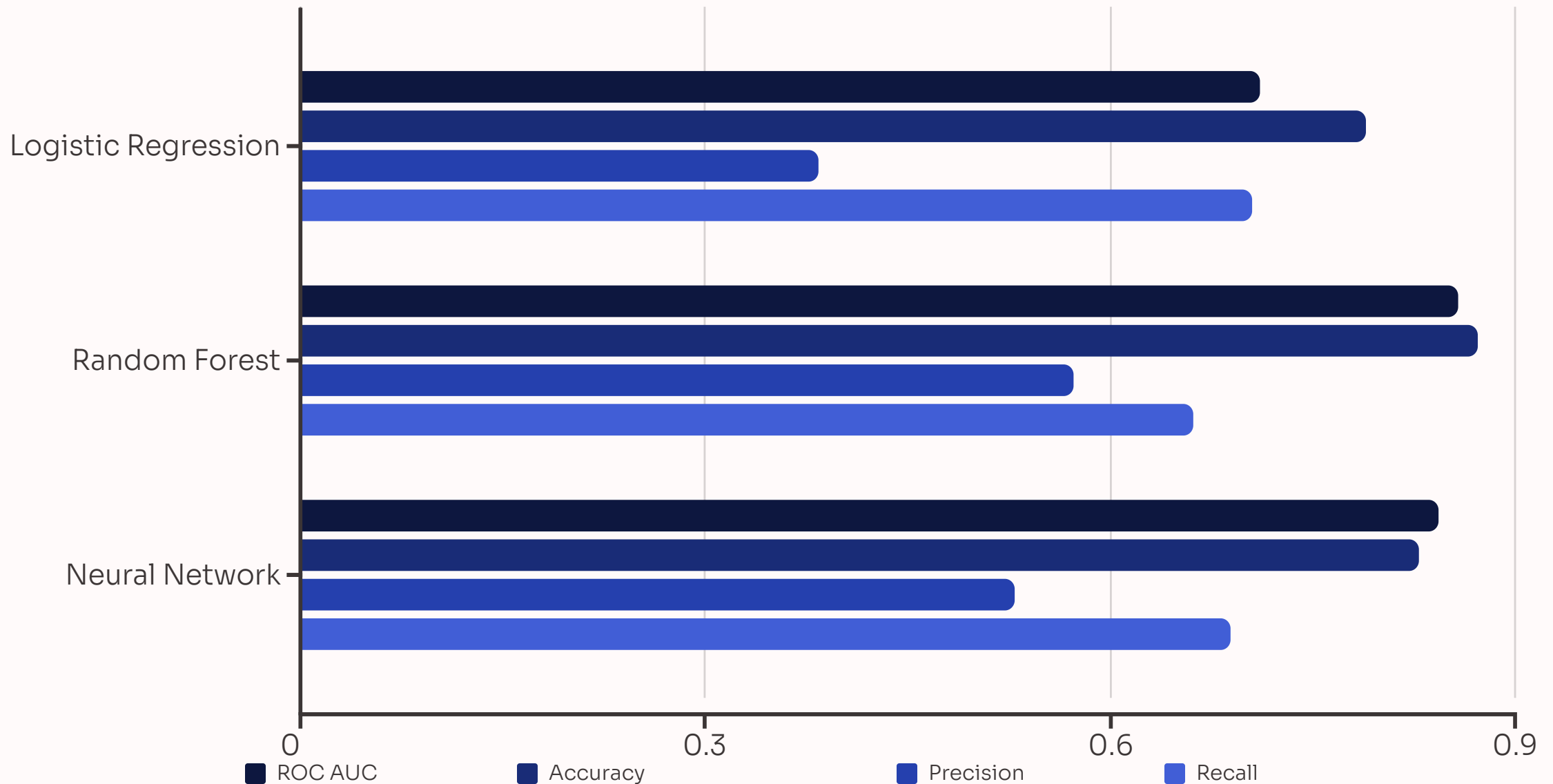


Conclusion

Neural Network performs similarly to Random Forest but lacks interpretability and is slightly less balanced. Might be useful if further tuned, but less ideal for business stakeholders.



Model Performance Comparison



The Random Forest Classifier emerged as the most effective model, achieving an impressive ROC AUC of 0.8580, outperforming both the baseline Logistic Regression and the extensively tuned Neural Network. This suggests that for this specific tabular dataset, the ensemble learning capabilities of Random Forest provided a more robust solution than deep learning.

Interpreting Churn Probability



Probabilistic Output

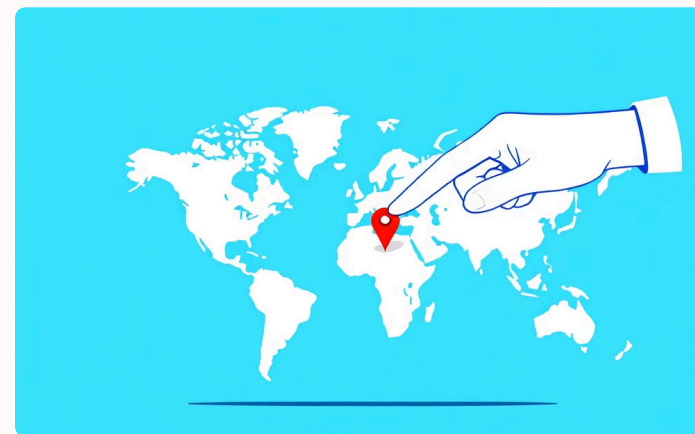
Models output a score from 0-1, representing the likelihood of customer churn. A score of 0.85 indicates an 85% chance of churn, moving beyond simple "yes/no" classification.

Probability scores enable dynamic threshold setting based on business needs—adjust the sensitivity depending on intervention costs and risk tolerance. This granular approach provides substantially more business value than binary predictions.



Risk Segmentation

Enables categorization into high risk (>0.8), moderate risk (0.5–0.8), and low risk (<0.5) customers, allowing for targeted retention approaches based on specific risk levels.



Resource Optimization

Allows banks to focus retention efforts on the highest-risk customers, maximizing the return on investment (ROI) from intervention campaigns by targeting resources efficiently.

Challenges Faced & Problem-Solving Approach



Class Imbalance Resolution

Our dataset suffered from significant class imbalance, risking biased model predictions. We addressed this by implementing SMOTE within the scikit-learn pipeline to generate synthetic minority samples, effectively balancing the dataset while preventing data leakage.

Both challenges were overcome through systematic approaches that balanced computational constraints with model performance objectives, significantly improving our ability to correctly identify potential churners.



Neural Network Tuning Efficiency

Optimizing the PyTorch Neural Network was complex due to numerous hyperparameters and high computational intensity. We overcame this with an iterative approach, leveraging GPU acceleration on an NVIDIA RTX 4060, and implementing early stopping for efficient training.

ChurnGuard: Interactive Demo

Our live demonstration showcases how ChurnGuard transforms sophisticated ML insights into actionable business intelligence, enabling proactive customer retention at scale.

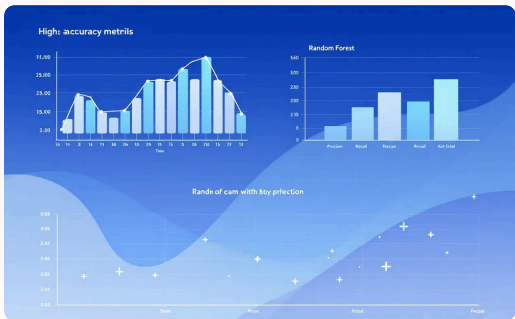
Conclusion & Future Work

Key Achievements:



Robust ML Application

Developed a robust machine learning application for credit card churn prediction, demonstrating high reliability and accuracy.



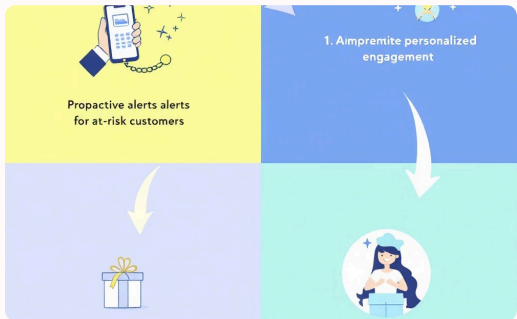
Optimal Model Performance

Identified Random Forest as the optimal model, achieving an impressive ROC AUC of 0.8580, superior to other tested algorithms.



Modular System Architecture

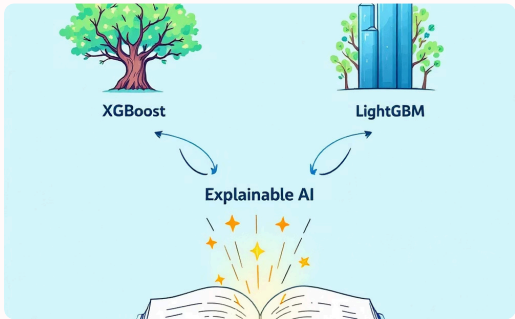
Created a modular system with a FastAPI backend and a Streamlit frontend, ensuring scalability and ease of maintenance.



Actionable Retention Framework

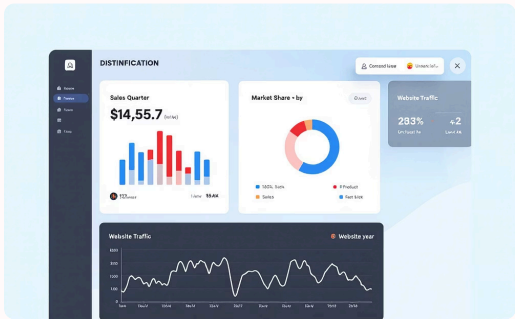
Provided an actionable framework for customer retention, enabling businesses to pro-actively engage at-risk customers.

Future Directions:



Advanced ML & Explainable AI

Explore advanced models like XGBoost and LightGBM, and incorporate Explainable AI techniques for greater transparency.



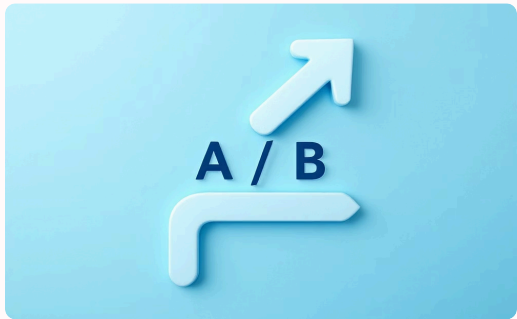
Enhanced User Experience

Implement detailed dashboard visualizations and an automated notification system for improved user insights and timely alerts.



Secure Cloud Deployment

Integrate user authentication and deploy the system to a secure cloud platform for robust, scalable operations.



A/B Testing Strategies

Conduct A/B testing of various retention strategies to continuously optimize and validate the effectiveness of interventions.